

CPEN 321 - M1: App Skeleton

Documentation and Reflections • Yichang Chen • September 2026

1. Development Info

1.1 Repository

GitHub repository	https://github.com/CYCRUOK/Yichang-Chen-CPEN321-M1
Submitted commit (main)	985bed7709f03a58499431e541e9b650c19c7e0a
Submitted APK	M1_CPEN321Application.apk - release build, signed with the developer's release key (SHA-1 49:51:EE:37:A4:56:5A:C8:26:D9:DD:AE:4B:F6:C5:3B:2E:09:D3:BB, registered as an Android OAuth client).

1.2 Back-end server

Public IP	8.229.19.28 (Google Cloud Compute Engine, us-west1, e2-small, Ubuntu 24.04, static IP)
Domain name	8-229-19-28.sslip.io (sslip.io resolves the IP encoded in the name; no DNS records to manage)
Base URL used by the APK	https://8-229-19-28.sslip.io - HTTPS with a Let's Encrypt certificate (valid until 2026-12-16, auto-renewed by certbot). HTTP on port 80 redirects to HTTPS.
Endpoints	GET /health' GET /api/server-ip' GET /api/server-time' GET /api/name' GET /api/ws-status', WebSocket wss://8-229-19-28.sslip.io/ws (relay of wss://8.229.22.124).
How it runs	nginx (TLS termination, WebSocket upgrade for /ws) → Docker Compose → Node 24 container running the compiled server (node dist/index.js), restart: unless-stopped; verified to come back by itself after a VM reboot.

1.3 Secrets, configuration parameters and setup steps for TAs

Everything below is also in the repository README.md. Nothing needs to be configured to grade the submitted APK against the running server - the values are already baked in.

Back-end (backend/.env, copy from backend/.env.example):

Variable	Value / meaning
PORT	3000
OWNER_FIRST_NAME / OWNER_LAST_NAME	Yichang / Chen - returned by GET /api/name

ME	
UPSTREAM_WS_URL	wss://8.229.22.124 - course pixel stream relayed on /ws
SERVER_PUBLIC_IP	optional override; when unset the server asks api64.ipify.org
JWT_SECRET · MONGODB_URI · GOOGLE_CLIENT_ID	present in the template, <i>not used</i> by MI (no database, no server-side token verification)

Run locally: `cd backend && npm install && npm run dev`, or `./scripts/run-backend.sh` (Docker Compose). Tests: `./scripts/run-backend-interface-tests.sh`. Cloud: `scripts/cloud-setup.sh <repo-url>` then `DOMAIN=... EMAIL=... scripts/cloud-https.sh` on a fresh Ubuntu VM with TCP 80/443 open.

Front-end (frontend/local.properties, copy from frontend/local.properties.example):

Property	Value / meaning
sdk.dir	path to the Android SDK (Platform 35 for compileSdk - AGP installs it automatically - and 36 for the emulator)
API_BASE_URL	https://8-229-19-28.sslip.io (the WebSocket URL is derived: wss://.../ws)
GOOGLE_CLIENT_ID	1051534582649-8v5q22t61qss82jog37uvgfop025tcg9.apps.googleusercontent.com - the <i>Web application</i> OAuth client ID used by Credential Manager (public, baked into the APK)
RELEASE_STORE_FILE ··· RELEASE_KEY_PASSWORD	optional release signing; without them assembleRelease signs with the debug key

Build: open frontend/ in Android Studio (Narwhal 3 / 2025.1.3 or newer, required by AGP 8.13) or `cd frontend && ./gradlew assembleRelease`. Run: `./scripts/run-frontend.sh` (expects an AVD named Pixel_9). Tests: `./gradlew testDebugUnitTest` (host) and `./scripts/run-frontend-e2e-tests.sh` / `./gradlew connectedDebugAndroidTest` (instrumented).

Google sign-in note. Google only issues an ID token to APKs whose signing certificate SHA-1 is registered as an Android OAuth client of the project. The submitted APK's key is registered. An APK you build yourself is signed with *your* debug key, so Google sign-in in it will fail with "Sign-in cancelled, or this build's signing key is not registered..." unless that SHA-1 is added to the project (I can add a TA's SHA-1 on request). Buttons 2 and 3 do not depend on this.

1.4 User accounts / credentials needed to exercise the app

- Button 1 needs any Google account on the device; the OAuth consent screen is published (External), so no test-user allow-list applies.
- No app-specific accounts, passwords or API keys are needed. The ISS position API (api.wheretheiss.at) and ipify are free and key-less.

1.5 Devices

The app runs on a **Pixel 9 emulator with Android Baklava (API 36)** and on a physical **FCNT arrows M08 (Android 15)**. It compiles against API 35 (pinned by the course template).

1.6 Button 3 - Timer and "While You Waited"

The user sets a timer in minutes and seconds. The countdown is derived from a fixed deadline (not from counting ticks), so it stays exact across rotation and while the app is in the background. When it goes off the phone vibrates, posts a "Time's up!" notification (Android 13+ requests the notification permission on the first Start), confetti falls over the screen and a "**While you waited N s...**" summary appears, telling the user what happened in the world during exactly the time they waited:

- **The International Space Station** - its live position is fetched from <https://api.wheretheiss.at> when the timer starts and again when it fires; the app computes the great-circle distance flown (haversine at the ISS altitude) and draws both fixes and the hop on a mini equirectangular world map with coarse continent outlines drawn from an in-app polygon table.
- **Earth** travelled $29.78 \text{ km/s} \times t$ along its orbit.
- **Light** travelled $299\,792 \text{ km/s} \times t$, expressed in Moon round trips.
- Whether the **sunlight** that left the Sun at Start (8 min 19 s away) has arrived yet.
- Approximate **heartbeats, blinks and babies born worldwide**.

If the ISS API is unreachable the card degrades gracefully and the locally computed facts are still shown. The surprise therefore combines an external API, the app's own computation, sensors/OS features (vibration, notifications) and an animation, and its content is tied to the timer duration itself.

1.7 Limitations

- Google sign-in works only for APKs signed with a registered key (submitted APK, or the developer's debug key); see 1.3.
- The name shown for the signed-in user is the Google account's profile name (given + family name). If the account has no family name set, only the given name is displayed.
- The ID token is not verified server-side; the three information APIs are public. This is acceptable for M1 but not for the project.
- Client IP shows the device's private Wi-Fi/mobile address (IPv4 preferred, IPv6 otherwise), not its NAT public address.
- The pixel stream has no "new image" marker; the canvas is cleared when the server pauses for more than 2 s. Joining mid-image shows a partial first image until the next pause.
- The timer runs inside the app process; if Android kills the app in the background, the alert is lost (no AlarmManager / foreground service).
- The ISS card needs internet; the ISS distance is an estimate from two position samples (or speed $\times$ time if only one sample was obtained).
- The continent outlines on the mini map are deliberately coarse hand-simplified polygons.
- The certificate is bound to `8-229-19-28.sslip.io`; if the VM's static IP ever changed, the domain, certificate and `API_BASE_URL` would all have to be regenerated.

2. Time and AI Reliance Distribution

ID	Task (a short description)	Time spent on the task (hours)	AI reliance: 0 - 100% (estimate)
1	Environment setup	0.5	0
2	Backend	2	60
3	Android skeleton	1	80
4	Google sign-in (Credential Manager) and Google Cloud OAuth setup	1	20
5	Button 1 information table	1	70
6	Backend WebSocket relay of the course pixel stream	2	80
7	Button 2 live pixel-art screen	1	50
8	Button 3 timer and "While You Waited" surprise	2	70
	Overall	10	53.75

3. AI Use Reflection

Task 1 -

Task description: Forward the pixel stream provided by the course server through the backend to the app, and render it pixel-by-pixel in real-time onto a 16×16 grid.

AI interfaces, tools and models used: Claude code Opus 5

Strategy: First, have the AI write a script to probe the characteristics of the information provided by the course server. Then, have the AI use ADB to install the app on a virtual machine and check if the images display correctly.

Advantages: I can quickly explore and understand details such as the course server's protocols.

Disadvantages: The pattern display and UI interface require visual verification.

Task 2 -

Task description: Deploy the backend to Google Cloud and use a trusted certificate to enable HTTPS/WSS for the app.

AI interfaces, tools and models used: Claude code Opus 5

Strategy: Use AI to write two idempotent scripts (`cloud-setup.sh` for Docker, cloning, `.env` setup, and Compose; `cloud-https.sh` for Nginx, Certbot, and WebSocket upgrades). Execute them via SSH and then performed validation: running the same local smoke test against the public IP, checking the certificate chain with OpenSSL, and rebooting the VM to verify self-recovery.

Advantages: One-click deployment can be achieved using reproducible scripts, eliminating the need to manually enter commands.

Disadvantages: Granting the AI full shell access to the VM could raise security concerns.

Was AI used to prepare this document?

No.